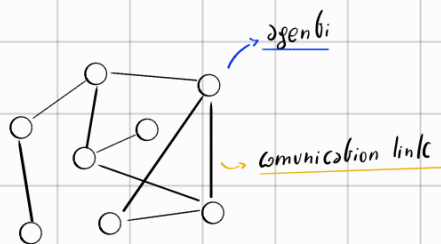


Modello distribuito

Un modello distribuito consiste in un insieme E di entità che comunicano fra loro tramite dei messaggi

Solitamente le entità comunicano fra loro per calcolare una soluzione ad un problema

La topologia di un sistema distribuito può essere rappresentata tramite un grafo del tipo



1.1. Entità agente o nodo

Ogni entità $x \in E$ è composta da

- Una memoria locale non condivisa M_x

All'interno di M_x è presente un insieme di registri fra cui troviamo il registro $state(x)$, il quale deve essere sempre presente e

ben definito $\leadsto \forall x \in E, state(x) \in \Sigma \leadsto$ insieme ben definito di stati

- Un processore in grado di eseguire computazioni locali
- Un clock interno

Operazioni eseguibili dall'entità

Un'entità può eseguire solo 4 tipi di azioni

- eseguire del codice presente in M_x $\leadsto$ con aggiunta di istruzioni send, receive
- possibilità di settare/resettare il clock interno
- trasmissione di messaggi
- cambiare i valori dei registri interni

Eventi

Il comportamento del nodo $x \in E$ è di tipo reattivo, infatti questo si attiva solo in risposta a degli stimoli chiamati eventi altrimenti rimane inerte

Gli eventi possono essere di tre tipi:

a) Arrivo di messaggi

b) Tick del clock

c) Attivazione spontanea

dispositivi I/O

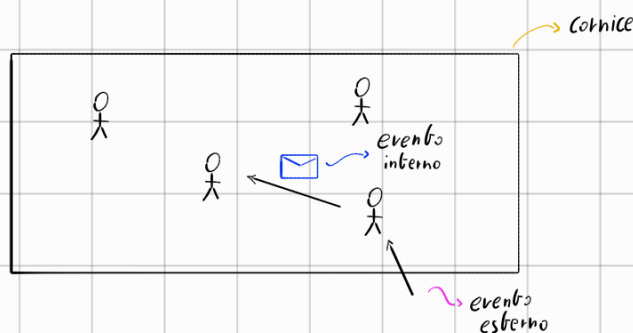
Definita la cornice come il "contenitore esterno" di un modello distribuito possiamo dividere gli eventi in 2 categorie:

- eventi interni

eventi scaturiti da componenti interne al sistema (Arrivo di messaggi e Tick di clock)

- eventi esterni

scaturiti da input provenienti da dispositivi connessi agli agenti (Attivazione spontanea)



Azioni

In risposta ad un evento e , un'entità $x \in E$ eseguirà una sequenza Sinistra ed indivisibile di operazioni chiamata azione.

- Finita nel senso che una volta iniziata questa terminerà in tempo finito
- Indivisibile (o Atomica) nel senso che una volta iniziata, questa deve necessariamente terminare

Behavior

L'azione eseguita dal nodo $x \in E$ dipende dalla tipologia dell'evento e dallo stato in cui il nodo si trova nell'istante in cui x riceve l'evento

Più nello specifico si indicherà con regola la tripla:

Status $\times$ Evento $\rightarrow$ Action

Il Behaviour (o comportamento) di un nodo x è

L'insieme $B(x)$ delle regole che guidano l'entità x

L'insieme $B(x)$ deve essere:

- Completo
 - Deterministico
- Coppia evento, stato | Azione possibile

Definiamo inoltre un collective Behaviour di un sistema $\mathcal{E}$ l'insieme

$$B(\mathcal{E}) = \{B(x) : x \in \mathcal{E}\}$$

In base a $B(\mathcal{E})$ possiamo definire il sistema come

- Simmetrico:

$$\forall x, y \in \mathcal{E} : B(x) = B(y)$$

- Asimmetrico:

$$\exists x, y \in \mathcal{E} : B(x) \neq B(y)$$

Proprietà 1.1.1

Ogni sistema Asimmetrico può diventare Simmetrico

→ Verrà visto come una sequenza di bit

→ Solitamente viene definito all'interno del sistema

1.2 Communication

In un sistema distribuito le entità comunicano fra di loro trasmettendo e ricevendo dei messaggi

Definiamo ora i seguenti insiemi

- $N_{out}(x) \subseteq \mathcal{E}$

e' l'insieme di entità a cui x può inviare dei messaggi chiamati out-neighbour di x

- $N_{in}(x) \subseteq \mathcal{E}$

e' l'insieme di entità da cui x può ricevere dei messaggi chiamati in-neighbour di x

Communication topology

Il grafo diretto $G(V, E)$ va a descrivere la topologia dell'insieme dei canali di comunicazione composto da:

- V insieme dei nodi
- $E \subseteq V \times V$ insieme degli archi c.c.

$$(x, y) \in E \Leftrightarrow y \in N_{out}(x)$$

1.3 Assunzioni

Finite communication delays

Viene assunto che, in assenza di "fault", i messaggi arrivano sempre in tempo finito garantendo la sua correttezza

Local orientation

Ogni entità distingue i propri in-neighbours e out-neighbours

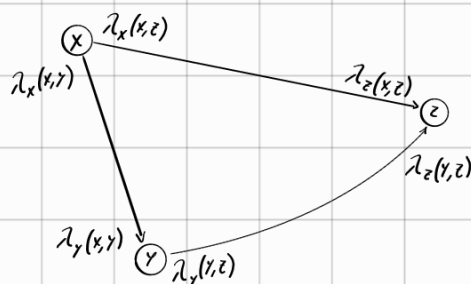
In altre parole ogni entità ha una funzione λ_x che associa delle etichette ai communication links

Quindi $\forall (x, y) \in E$ ci sono due etichette

- $\lambda_x(x, y)$ associata ad x
- $\lambda_y(x, y)$ associata ad y

A causa di questo assioma avremo che fare con un

grafo orientato etichettato (G, λ) con $\lambda = \{\lambda_x : \forall x \in V\}$



1.4 Restriction

Per risolvere un problema, erogare un servizio, etc..., un protocollo può essere pensato per un sistema distribuito con diverse proprietà aggiuntive.

Aumentare il numero di proprietà del sistema però limita l'applicabilità del protocollo stesso

In sintesi è fondamentale esplicitare tutte le restrizioni aggiuntive

Communication restriction

Parliamo ora delle restrizioni relative alle comunicazioni tra le entità

- Message ordering

In assenza di Saltimenti, i messaggi trasmessi da un'entità allo stesso out-Neighbour arriveranno a destinazione nello stesso ordine in cui vengono inviati. $\rightarrow$ Logica FIFO

- Reciprocal communication

$\forall x \in E \quad N_{in}(x) = N_{out}(x) \rightarrow$ Stiamo dicendo che se $(xy) \in E$ allora anche $(yx) \in E$

- Bidirectional links

$\forall x \in E \quad N_{in}(x) = N_{out}(x) \text{ AND } \lambda_x(x, y) = \lambda_x(y, x)$

Se vale il bidirectional links il grafo può essere semplificato come:



Reliability restriction

Passiamo ora alle restrizioni riguardanti l'affidabilità dei communication links

- Guaranteed reliability:

Qualsiasi messaggio inviato arriverà sempre non corrotto

- Partial reliability:

$\rightarrow$ da vedere se il tempo è definito

Da un dato istante in poi i messaggi arrivano sempre a destinazione

$\rightarrow$ Solo nel futuro

- Total reliability

$\rightarrow$ Anche nel passato

Non c'è mai avvenuto né avverrà mai che nel sistema ci sia stato un fault

Topological restriction

In genere un'entità non è connessa a tutte le altre tuttavia è stata introdotta la seguente restrizione

- Connectivity:

La topologia della rete di comunicazione $G=(V, E)$ è un grafo fortemente connesso

Time restrictions

In generale non si possono fare assunzioni riguardanti il tempo, per questo sono state introdotte delle restrizioni sui ritardi dei messaggi

- Bounded communication delays:

in assenza di problemi

Esiste una costante Δ t.c. il ritardo dei messaggi è $\leq \Delta$

- Unitary communication delays:

In assenza di fault il communication delay di ogni messaggio è di un'unità di tempo

- Synchronized clocks:

Il clock assume lo stesso valore per ogni agente nel sistema

Performance analysis

A differenza dei sistemi centralizzati la qualità di un protocollo è misurata tramite

- Message complexity

Ovvero il numero in termini di complessità asintotica di messaggi scambiati

- Time

Dato una definizione di tempo globale calcolato come il delay tra l'inizio della computazione sulla prima macchina e il tempo in cui l'ultima entità la termina

Tempo di Germinazione e Tempo di completamento

All'interno di un sistema distribuito e' essenziale la differenziazione di questi due tempi:

Def

Con il tempo di Terminazione indichiamo
l'istante in cui $\forall x \in E$ si trova in uno stato quiescente

Def

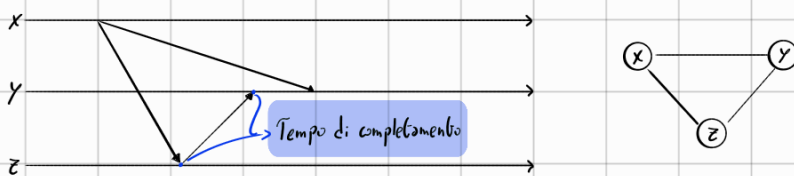
Con il tempo di Completamento indichiamo
l'istante in cui $\forall x \in E$ si trova in uno stato quiescente ed il protocollo ha completato la propria task

Stati Transitori e ritardi

Nonostante un protocollo sia deterministico, a causa dei ritardi tra l'invio e la ricezione dei messaggi,

il numero di stati transitori cresce esponenzialmente

Assumere che il protocollo sia completo implica che questo debba sempre portare ad uno dei possibili stati finali



L'avversario gestendo i ritardi può permutare l'ordine in cui i nodi vengono in formato

Pero', assumendo che il protocollo sia completo, $\exists T_{\text{com}}$ in cui il protocollo termina

Tecniche per dimostrare i Lowerbound

Per la dimostrazione dei Lowerbound nei Sistemi distribuiti si passa sempre tramite gli Environment

Def

Con Environment si indica l'insieme

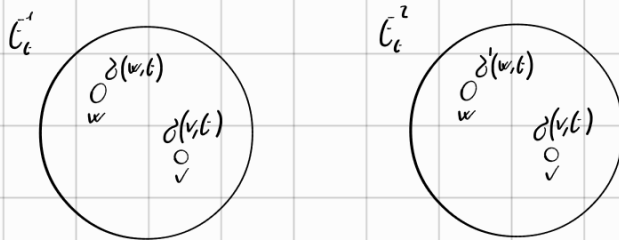
$$\mathcal{E}_t = \{ \sigma(x, t) : \forall x \in \mathcal{E} \}$$

Dove con $\sigma(x, t)$ si intende lo stato in cui si trova x al tempo t

Più nello specifico prendiamo le seguenti proprietà degli environment

Sia $v, w \in \mathcal{E}$

Siano $\mathcal{E}_t^1$ e $\mathcal{E}_t^2$ 2 environment t.c.



Allora v non capisce se si trovi in $\mathcal{E}_1$ o in $\mathcal{E}_2$

Più nello specifico abbiamo che

1) Se l'evento e accade nello stesso istante t in $\mathcal{E}_1$ e in $\mathcal{E}_2$ dove $\sigma_1(v, t) = \sigma_2(v, t)$

Allora il nodo v eseguirà la stessa azione sia in $\mathcal{E}_1$ che in $\mathcal{E}_2$

2) Se x, w in $\mathcal{E}_1$ si trovano nello stesso stato $\sigma(v, t) = \sigma(w, t)$ e ricevono lo stesso evento

Allora x e w eseguiranno la stessa azione